Development Plan
Lake

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

Table 1: Revision History

Date	**Developer(s)**	**Change**
September 27, 2026	Colin Chambachan	Introduction and Sections 8 to 10 (Project Decomposition and Scheduling, Proof of Concept Demonstration Plan, Expected Technology)
September 27, 2026	Matthew Nesbitt	Sections 1 to 7 (Confidential Information through Workflow Plan)
September 27, 2026	Jinwoo Hong	Generative AI Use Disclosure and Reflection appendices
September 28, 2026	Ethan Walsh	Team Charter appendix
September 28, 2026	Gary Qin	Sections 11 and 12 (Use of AI and LLMs, Coding Standard)
September 28, 2026	Matthew Nesbitt	Added quality assurance role, fixed typos and removed a template comment
October 2, 2026	Colin Chambachan	Added a direct link to the GitHub repository in the introduction (peer review issue #33)

This document serves as the development plan for Lake by CinqGarçons. Lake splits a large language model (LLM) across multiple devices so that they can run the model together. All source code, documentation and issue tracking for the project live in the Lake GitHub repository (https://github.com/capstoneCEGJM/Lake). The plan covers how the project will be conducted. The first sections deal with confidentiality, intellectual property and licensing, followed by how the team is going to be organised for the duration of the project. Section 8 breaks down the work and lays out the schedule, Section 9 describes the proof of concept, and Section 10 lists the expected technology. The plan finishes with the use of AI tools and coding standards, while the team charter and reflections are in the appendices.

1 Confidential Information

There is no confidential information to protect.

2 IP to Protect

There is no intellectual property information to protect.

3 Copyright License

Lake is licensed under the GNU General Public License Version 3. A copy of the GNU GPL license is available in the Lake repository here.

4 Team Meeting Plan

CinqGarçons will meet at minimum weekly every Friday at 2:40PM EST via Discord. Additional ad-hoc meetings will be scheduled as the need arises. Some ad-hoc meetings may be in-person, likely after/before lecture. There will be a minimum requirement of one in-person meeting per month. More than one meeting per month may be in-person, and it is up to the discretion of the team to decide when to convert a virtual meeting to an in-person meeting.

Each week, team members will take turns being the chair and leading the meeting. The chair will be responsible for bringing a formal agenda to each meeting. The chair will also be responsible for introducing the agenda to the team, and ensuring the agenda points are addressed during the meeting. Ad-hoc meetings may only have an informal unwritten agenda due to their potentially last-minute nature.

The order in which members will fill the role of meeting chair are as follows:

- Colin Chambachan
- Jinwoo Hong
- Matthew Nesbitt
- Gary Qin
- Ethan Walsh

5 Team Communication Plan

Formal text-based team communications will be facilitated through a Discord server, with additional communication happening verbally during meetings.

Project design and repository related communications may also occur on a pull request during the review phase, or in the comments of a related GitHub issue. All formal action items shall have an associated GitHub issue to track the item's progress. Pull requests should be associated with an issue, and may resolve said Issue. Issues will be the primary method of work assignment and work splitting within the team, as each issue shall have one or more assigned members. Issues may also be created as a discussion thread for particular design decisions that require deliberation.

6 Team Member Roles

Team members may fill roles dynamically throughout the year as best fits the situation. Some roles, however, are assigned permanent members. Members may fill more than one role, and roles may be filled by more than one member. Quality Assurance role is dynamic and follows the same rotation as Meeting Chair. Technical leads were assigned while writing the Software Requirements Specification (SRS). The list below matches its Roles and Personnel section. Identified member roles are as follows:

- Liaison: Ethan Walsh
- Meeting Chair: Dynamic
- Quality Assurance: Dynamic
- Repo/Org Administrator: Matthew Nesbitt
- Technical Lead (Architecture): Ethan Walsh
- Technical Lead (Coordinator): Gary Qin
- Technical Lead (ML): Jinwoo Hong, Colin Chambachan
- Technical Lead (Performance): Matthew Nesbitt

7 Workflow Plan

All contributions to production are required to go through a pull request, and aquire approval from at minimum two reviewers. When working on a contribution, using either the branching workflow or forking workflow is acceptable, and it is up to the individual contributor to decide which workflow to use.

7.1 Contribution Flow

1. For branching workflow:
   (a) Create a new working branch in the main repository. The branch name should be descriptive, and should ahere to the conventional-branch standard.
   (b) Commit desired changes to the working branch. Commit names should be descriptive but turse, and should adhere to the conventionalcommits standard.
2. For forking workflow:
   (a) Create a fork of the main repository. Only one fork per contributor is required.
   (b) Create a new working branch in this fork. There are no restrictions on branch names made in forks.
   (c) Commit desired changes to the working branch. Commit names should be descriptive but turse, and should adhere to the conventionalcommits standard.
3. Open a pull request from the working branch to the `main` branch in the main repository.
4. Address any changes requested during review.
5. After the pull request has recieved two approvals and passes CI, it can be merged.
6. For branching workflow:
   (a) After a PR has been merged, the associated work branch should be deleted. This can be done directly from the PR page.

7.2 Issues

Issues will be used to track action items, and occasionally for design decisions that require deliberation. When creating an issue, a description should be included providing all the information required to address the issue. In addition, the issue may be labelled with one or more relevant tag(s) if applicable. The available tags are as follows: `bug`, `feature`, `decision`, `improvement`,

`priority: high`, `priority: low`. Pull requests should be associated with an issue, and may close said issue by including a close tag.

Issues will also be created (in separate projects) for each lecture, as well as each TA meeting. Issue templates will be created for both lecture and TA meeting issues to make the creation of these issues easier and faster.

7.3 CI/CD

CI/CD will utilise GitHub Actions, will be required to pass for a PR to be merged, and will consist of the following stages:

- PDF building and publishing
- Code linting
- Unit testing
- Integration testing
- System testing
- Performance testing

8 Project Decomposition and Scheduling

All planning runs through GitHub, and the schedule follows the course deadlines, each with an internal deadline two days earlier.

8.1 GitHub Projects

All project work is tracked on the Lake GitHub Project. Every issue from the Workflow Plan gets a card on the board, and cards move through Backlog, Ready, In Progress, In Review and Done columns. An issue only moves to Ready once it has a description, a work estimate and at least one assignee. Cards are also tagged with the part of the system they touch, such as the Coordinator, the Lake Client, networking or documentation.

Each deliverable in Table 2 gets its own GitHub milestone, with the due date set to the internal deadline rather than the course deadline. That way the board always reflects the date being worked towards. Lecture and TA meeting issues are kept in separate projects, as noted in the Workflow Plan, so this board only holds project work.

8.2 Breaking Down Work

Work is split by feature instead of by document. On the code side, issues map to parts of the system. On the documentation side, an issue is typically a section or one piece of analysis. Anything that looks like more than about eight hours of work gets split before anyone starts on it.

Every issue should say what done looks like. For code, that is usually a test that passes or a measurement that gets recorded. For a document section, done means the section meets the level 4 rubric criteria and has been checked against the deliverable's checklist.

The course assumes about ten hours per person per week, including class time, which gives roughly 50 hours of team time a week to plan around. At the Friday meeting, the chair goes through the board with the next internal deadline in mind. Members pick up Ready issues, and the chair makes sure nobody is overloaded while someone else has nothing to do. Members shall also take issues outside their own technical lead area now and then, so that more than one person understands each component.

8.3 Schedule

The team's schedule is built around the given course deadlines while allocating an internal deadline two days before each public facing one. Reserving the last two days allows for double-checking deliverables and catching issues that slip through. For demonstrations such as the proof of concept (PoC), the internal deadline is two days before the assigned slot, and after that point only bug fixes are merged. Table 2 lists every graded deliverable, with dates taken from the course calendar. Smaller items like the contribution surveys, the productivity reports and the module implementation exercise go on the board with the same two-day buffer.

Implementation runs alongside the documents in the phases shown in Table 3.

Table 2: Course deliverables with official and internal deadlines

Deliverable	**Weight**	**Official**	**Internal**
Fall term, 2026			
Problem Statement, PoC Plan, Development Plan and Team Charter	2%	Sept 28	–*
Peer review of another team's Problem Statement and Development Plan	–	Oct 1	Sept 29
SRS and Hazard Analysis (Rev 0)	7%	Oct 19	Oct 17
Peer review of SRS and Hazard Analysis	–	Oct 22	Oct 20
V&V Plan (Rev 0)	5%	Nov 2	Oct 31
Peer review of V&V Plan	–	Nov 5	Nov 3
Design Document (Rev −1) and Design Thinking Report (CD-Fall)	3%	Nov 16	Nov 14
Peer review of Design Document (Rev −1)	–	Nov 19	Nov 17
PoC Demonstration and interviews	5%	Nov 23 to Dec 3	†
Winter term, 2027			
Design Document (Rev 0)	4%	Jan 18	Jan 16
Peer review of Design Document (Rev 0): implement one of another team's modules	–	Jan 21	Jan 19
Rev 0 Demonstration and interviews	10%	Feb 1 to 11	†
V&V Report (Rev 0) and Performance Report (CD-Winter)	4%	Mar 8	Mar 6
Peer review of V&V Report and CD-Winter	–	Mar 11	Mar 9
Final Demonstration (Rev 1)	20%	Mar 15 to 21	†
Final Documentation (Rev 1), EXPO poster and video	30%	Mar 31	Mar 29
Capstone EXPO	10%	Apr 6 (TBC)	Apr 4

*The internal deadline policy was adopted while this deliverable was already in progress.

†Two days before the team's assigned slot.

Table 3: Implementation phases

Phase	Dates	Main work
Setup	Sept 28 to Oct 16	Get ggml and llama.cpp building on every team member's machine and record single-device baselines
Proof of concept	Oct 19 to the PoC slot	The two-device split described in Section 9
Core system	December	Any number of devices, profiling, the Model Partitioner, Lake Clients joining and leaving, and re-splitting the model between requests
Dynamic allocation	January to the Rev 0 slot	Re-splitting during a session, and the Lake Client
Performance	Feb 12 to Mar 5	Benchmarks for the Performance Report, and the wireless stretch goal
Refinement	Mar 8 to Apr 6	Feedback from Rev 0, final documentation and EXPO preparation

9 Proof of Concept Demonstration Plan

The PoC is meant to show early on that the project is feasible and to establish the scope of what shall be delivered by the end of the project.

9.1 Risks

The biggest risk is not being able to split a model across devices at all. Tools like llama.cpp run the whole model in one process, so splitting it means working directly with ggml, the tensor library underneath. Each device has to load only its own range of layers from the model file, run them, keep its own key-value (KV) cache, and pass the hidden state on to the next device intact. Every other part of Lake depends on this working.

Two smaller risks come along with it. The first is network overhead. Text is generated one token at a time, and each token has to cross every device connection. One transfer is small, but its latency is added to every token. On a slow link, a split model could end up noticeably slower than running it on one machine.

The second risk is mixing platforms. Kernels differ across device types, so their floating-point results are not bit-identical. Devices on different platforms have to agree on the hidden state format, and the output has to stay coherent despite the small numerical differences.

Dynamic re-allocation is what sets Lake apart from existing tools, but it is left out of the PoC. Re-allocation cannot work until the basic split does, and

trying to do both by November would turn the PoC into a prototype.

The PoC is also not expected to be faster than a single device. With a pipeline split, each token still goes through every layer in order, so splitting a model that already fits on one machine does not speed up a single request. Splitting pays off when a model is too big for any one device, or when several requests run at once. For the PoC, the goal is to measure how much overhead the split adds.

9.2 Demonstration

The demonstration uses a model (which is to be determined) to be split by hand down the middle. Initially, a model will be split up and run on two separate processes on one system to prove feasibility, and it will be further iterated on if feasible. A bare-bones command-line Coordinator hands out the partitions. The Model Partitioner, profiler and Lake Client are not part of the PoC.

The demonstration builds up in three steps, with each step adding one more thing that can go wrong:

1. Two processes on the same laptop, talking over localhost. This tests the split by itself, without a real network or different platforms.

2. Two laptops of the same type connected by a Thunderbolt cable, which adds real transfer latency. Wired Ethernet is a target for larger Lake networks.

3. Two laptops with different operating systems and processor architectures, for example an Apple Silicon MacBook and an x86 Windows or Linux machine.

Each step runs a prompt from the command line and compares the streamed output with the same prompt run through plain llama.cpp on one machine. Each device's memory use is also shown, to confirm that it loaded only its own layers, along with a per-token breakdown of compute time on each device against time spent on transfers.

The PoC counts as a success if the split model produces a sensible response to a prompt, with the hidden state passed between the two halves without errors. Reaching this in step one is enough. Steps two and three are attempted if time allows, and any progress on them is shown as extra evidence. The output does not need to match a single device exactly, and there is no speed target. Transfer times are recorded to guide later work, but they do not decide whether the PoC passes.

9.3 Fallback Options

If the PoC shows that one of these risks is worse than expected, the scope will be discussed with the TA afterward. The current fallback ideas are:

- If a custom split on top of ggml cannot be made to work, build Lake's coordination and allocation on llama.cpp's remote procedure call (RPC) backend instead.
- If network overhead is too high, move the performance goal towards models that do not fit on one device and towards several requests at once, since splitting still helps there.
- If splits across platforms produce bad output, limit Rev 0 to the platform combinations that work and make the rest a stretch goal.

10 Expected Technology

These are the current technology choices, which are subject to change once building starts. The SRS and design documents will not depend on them. Alongside the tools below, git and GitHub are used for version control, issues, pull requests, the project board and document publishing, and Discord for day-to-day communication.

10.1 Languages and Frameworks

The Coordinator will be written in Python. Its job is handing out layer assignments and collecting measurements, so it is not on the per-token path and Python's speed is not a concern there. Go and Rust were discussed, but the extra speed would not help in that part of the system, and the whole team is already comfortable with Python. The client-facing application programming interface (API) will use FastAPI, and the Lake Client will be built with React and TypeScript.

The Inference Engine on each device is expected to be written in Rust. The ggml library is written in C, and splitting the model means building ggml's compute graph directly. Rust matches C++ in speed and calls C functions directly, so there is no extra layer on every token. Rust also enforces memory safety and resource cleanup at compile time, where C++ leaves both to the programmer, which makes it easier to learn for a team that has mostly not worked in C++. The calls into ggml still need unsafe Rust, so that code is kept in one small module. C++ was the other option considered, since llama.cpp is written in it. If Android support happens, the Lake Client there would be written in Kotlin and call the same Rust code through the Java Native Interface (JNI).

Devices talk to each other and to the Coordinator using gRPC with Protocol Buffers. From one message definition, matching Python and Rust code is generated, using the Tonic library on the Rust side, and gRPC's streaming calls fit the token-by-token handoff. The downside is some framing overhead compared with raw sockets, which the PoC will measure. If the overhead turns out to matter, the alternatives to evaluate are raw TCP with FlatBuffers, ZeroMQ and QUIC, and QUIC would also help on unreliable Wi-Fi.

10.2 Inference and Models

The model math itself will run on ggml, the tensor library underneath llama.cpp. The library already has fast quantised kernels for both x86 and ARM, plus GPU backends such as Metal and CUDA, which covers every target platform. Lake's own code handles the splitting, networking and profiling around it. The catch is that ggml's API is low-level and changes often, so ggml will be pinned to a specific version.

Another option considered was ONNX with ONNX Runtime. ONNX models are portable, and standard tools can cut an ONNX graph into subgraphs, which would suit layer splitting. The deciding factor for ggml was that quantised versions of popular open models are much more widely available as GGUF files. ONNX Runtime stays as the backup if ggml's API proves too hard to work with.

Lake is meant to be model agnostic. Any pre-trained transformer LLM that ggml supports should work, rather than one specific model, and no model will be trained or fine-tuned. Small models are used during development because they load quickly, and larger ones are used later to test running a model that does not fit on one device. Weights are stored as 4-bit quantised GGUF files, which are roughly a third of the size of 16-bit weights. Each device memory-maps the file and reads only its own layers. Weights are downloaded under each model's own license and are not committed to the repository.

10.3 Components Built for Lake

Existing projects such as exo, prima.cpp and llama.cpp's RPC backend can already split models across machines, and they will serve as references. Lake still needs its own Coordinator, Model Partitioner and transport between devices. In exo, layers are assigned once at the start of a session, and llama.cpp's RPC backend fixes the split when the model loads. Changing the split during a session, which is Lake's dynamic allocation goal, needs control over layer assignment and the KV cache that these tools do not offer.

The Model Partitioner will profile each device's per-layer speed, free memory and network latency, then pick the split points that give the lowest time per token. A simple greedy approach comes first, with dynamic programming replacing it once that works.

10.4 Testing and Continuous Integration

Table 4 shows the linters and unit testing frameworks for each language. Code coverage on the Python side will use pytest-cov. On the Rust side, cargo-llvm-cov and tarpaulin both still need to be tried with the build before picking one. A coverage target will be set during the PoC phase, once there is enough code to measure.

GitHub Actions already builds and publishes the LaTeX documents, and a second workflow for code will run the stages listed in the Workflow Plan on

Table 4: Linters and unit testing frameworks by language

Purpose	**Python**	**Rust**	**TypeScript**
Linting and formatting	Ruff	Clippy, rustfmt	ESLint, Prettier
Unit testing	pytest	cargo test	Vitest

every pull request. The Rust build and tests will run on Linux, macOS and Windows runners, so anything platform-specific breaks right away rather than at a demo. For integration tests, continuous integration (CI) will split a tiny test model across two local processes and compare the result with an unsplit run, since a real model would take too long on every pull request. CI runners are shared virtual machines with noisy timing, so the performance stage there is only a smoke test for large regressions. Real performance numbers and system tests will come from team members' own machines before each demo.

10.5 Performance Measurement

Most performance data will come from Lake's own per-token profiler, which records compute time on each device and transfer time between devices. The same data feeds the Model Partitioner and the Lake Client. For comparison, llama-bench and llama-perplexity from llama.cpp give single-device numbers for speed and output quality. The unsafe code that calls into ggml will also be tested under AddressSanitizer in CI to catch memory bugs early.

11 Use of AI and LLMs

11.1 AI Usage

Each member of the team will use LLMs for code generation to different extents. For members who use LLMs most extensively, LLMs will be used to write the majority of the code for the project. Coding tasks that LLMs will primarily be used for include boilerplate code, unit tests, & in-line comments. LLMs will also be consulted as a second opinion for code reviews and verifying & considering alternatives to architectural design decisions & critical sections of the code.

11.2 AI Tools

The team will be using commercially-available Anthropic and OpenAI models (eg. Claude Opus/Sonnet, ChatGPT).

11.3 AI Verification

The team will verify the output of LLMs by ensuring all code is read & reviewed by the LLM user & at least 2 other members in a pull request. Due to the heavy usage of LLMs, Git commits will be atomic & pull requests will not exceed

400 lines of changes to ensure every PR can be reasonably reviewed carefully. Additionally, unit tests will be used to verify all code correctness, and code linters will be used to verify all code styling and syntax.

12 Coding Standard

The team will adopt industry-standard language-specific coding standards & styles guides. For Python code, the Black style guide and the Ruff linter will be used. For Rust code, the rustfmt formatter and Clippy linter will be used. For JavaScript & TypeScript files, ESLint and the Prettier formatter will be used.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

ChatGPT Web and Claude Web were used in conjunction with team member discussions to brainstorm ideas for sections of the development plan.

13.2 How and Where Used

Claude Web was used while researching and brainstorming technical details for the Proof of Concept Demonstration and Expected Technology sections. Research included how to split up the layers of an LLM accross devices, how `llamma.cpp` and `ggml` compares to `ONNX` for inference, whether to use Rust vs C++ for the device engine, how communication between devices would function, and how to test the split model.

ChatGPT Web and Claude Web were also used to provide a general pass throughout the document for any spelling or grammar issues.

13.3 Verification of AI-Generated Content

Proposed changes in the document are reviewed through pull requests, and the branch rules require at least two approvals from team members before merging. High level technical details were verified against `ggml`, `llama.cpp`, and `Tonic` documentation. However, claims about the feasibility of layer splitting, cross-device compatibility, and performance remain hypotheses until a proof of concept implementation can be used to verify them.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?

   It is important to create a development plan prior to starting the project, as it helps establish the high level project scope, team dynamics, and expectations.

   It is also important to establish an agreement in confidential information, IP, and licensing before a project is begun, as any amendments to it once it has been started may pose difficulties, especially once previous releases or proof of concepts have been created.

   Additionally, this project introduces many technical risks that can affect the feasibility and utility of the completed product (e.g. the feasibility and performance costs of distributing inference), and a discussion of risks and success criteria in the proof of concept demo help ensure that project risk is minimised as early as possible.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

   CI/CD benefits a team environment by standardising coding practices, ensuring lints, formatters, and tests are all met/passing before they are merged into the main branch. Automated tests also benefit by running in a reproducible and clean environment, including cross-platform tests that are important to ensure functionality across devices. The CI/CD can also be configured to automatically create artefacts (e.g. test builds), which can be used for additional validation, including performance benchmarking, or end-to-end regression testing.

   Disadvantages include maintenance overhead of tests and CI, including flaky tests or long running tasks, which may block valid changes or unnecssarily reduce team velocity. Standard CI runners may also lack the hardware capabilities to run certain jobs, and suitable runners may incur costs.

Many of these disadvantages can be circumvented with good CI/CD design and team-coordination; ensuring tasks relevant to the diff are ran in the CI, and balancing the frequency of longer running tasks.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

   Disagreements occurred in the underlying technologies and frameworks to utilise for the proof of concept, such as whether to use C++ or Rust for the Inference Engine. C++ was the initial choice due to being the language used in `llamma.cpp`, but Rust was suggested after it was determined that it can call `ggml`'s C interface, as well as provide other benefits, such as stronger memory checks, and better team familiarity. It was agreed to revise the plan to use Rust as the initial choice, while leaving final decisions open to change during implementation of the proof of concept.

Appendix — Team Charter

External Goals

13.3.1 Capstone EXPO Performance

The team aims to win a prize at the Capstone EXPO, but more importantly, provide an intriguing and interesting demo and presentation that attendees will find memorable.

13.3.2 Capstone course Performance

The team aims to receive a high mark (A+) in the Capstone course to reflect the effort that will go into the project.

13.3.3 Learning

Each member of the capstone team has expressed that a top priority for the course is learning new technical skills and technologies through the project. This spans to beyond technical skills, for qualities such as leadership, teamwork, and public speaking.

13.3.4 Have Fun

This course is one of the final opportunities for the team to work together and create an impressive final product. The journey should be a fun challenge for the team.

Attendance

Expectations

The team expects all members to attend a minimum of 80% of weekly scheduled team meetings. This does not apply to supplementary ad-hoc meetings. Leaving early is admissible if the member has another commitment (academic or extra-curricular). Missing meetings is admissible as long as the member provides a reasonable excuse to the team as early as possible. This policy for missing meetings applies to a given limit. If a team member misses 2 consecutive meetings, a mediator from the team will reach out to the member in question to try and resolve the attendance issue and determine the best next step.

Acceptable Excuse

An acceptable excuse is when external factors are involved (outside of usual academic and extra-curricular deadlines for other commitments). External factors would be considered family commitments, medical, or other serious issues.

Unacceptable excuses are those without reason or those involving other course deadlines. As upper year students it is expected for us to manage many

deadlines across multiple courses and other course deadlines are not a typically acceptable reason to be late on capstone work. In the case that work is projected to finish late, ensure other team members are aware so that they may step in to assist if available.

In Case of Emergency

Team members will try their best to inform other team members of their absence. The team also acknowledges that this is not always possible in an emergency situation.

Accountability and Teamwork

Quality

It is expected that all team members stay up to date on project happenings and the agenda for an upcoming meeting. Team members are considered responsible for bringing themselves up to speed on current decisions and actions so that they can contribute effectively.

In terms of deliverables, it is expected that documents are reviewed for clarity, grammar, and spelling. In addition, the document should be in alignment with level 4 of the corresponding rubric to maximise the team's mark. For code deliverables, it is expected that the code will pass the CI quality gate as well as all test cases (including the test cases that have been added for the new feature).

Attitude

The expectation for attitude is as follows: maintaining a co-operative, kind, and open attitude will result in the best final product. During meetings, it's important not only to arrive prepared, but also be an effective participant and listener who acts respectfully at all times.

With regards to feedback, always approach it from the philosophy that constructive feedback is beneficial in the long run and should not be taken personally. Those who are giving feedback should not be unnecessarily critical or attacking and should instead focus on areas for improvement.

13.3.5 Conflict & Feedback Resolution Plan

In the case of conflict, a team member uninvolved will act as a mediator between the two parties. In addition to both parties turning away from aggression, hostility, and negative comments, the mediator must ensure that both parties are adhering to this nature in an unbiased manner and the mediator must show empathy to both parties. The mediator may also decide to schedule a follow up to ensure all parties are able to continue discussing a conflict in a healthy manner or to ensure that it has been resolved.

Stay on Track

The primary method used to keep the team on track will be GitHub issues. GitHub issues allow the team to track a variety of deadlines and components of the project, including meetings, code tasks, and lectures. By assigning team members to issues within the GitHub project, the team is able to guarantee a roughly equal split of work. Due to the variable nature of the project and unknown targets for total code and commits, no target metrics will be set for commits at this point. Instead, we aim to measure participation via the code review process (comments and recommendations on pull requests). This way, the team can ensure that members are staying up to date with code changes they did not write, while also contributing to improving the quality of the overall product. Each pull request should have a minimum of two reviewers, and the split of total reviews should be roughly 20% per team member.

As stated earlier in the team contract, the team is setting a target metric of 80% attendance to all team meetings.

In the case that a team member does not meet their target metrics or not contribute their fair share and the reason is unknown or unacceptable, the team will host a meeting to find the root cause of the imbalance. In the case that further accommodations are required (e.g. a team member is facing a significant challenge outside of school), the issue will be raised upwards to be handled appropriately. Otherwise, the team will try to reach a consensus. For example, in order to try and even out the share of work, the team member in question may take on a greater share of the next deliverables and tasks to balance out workload in the long run. Because the team members have been friends for years, the team aims to solve any imbalances in a cordial fashion while keeping in mind that the work split at a specific time may not be equal as commitments vary. The expecation, however, is that in the long term the contributions average out and become roughly equal.

There is currently no specific reward set for team members who go above and beyond. A Potential reward is the team member not having to contribute to their share of the next team social event as a "thank you" from the rest of the team.

Team Building

The team will aim to spend time together that is not focused on capstone at a minimum of once per month. The team began this tradition in September by meeting for a dinner on the first weekend of the semester. Suggestions for further team building events include an escape room, board games, or seeing a game of one of the two new professional hockey teams in Hamilton.

Decision Making

Whenever the group is faced with a decision, a consensus strategy will be used. The consensus strategy allows each group member to share their opinion. In the ideal case, all team members are able to arrive at the same conclusion or

compromise if necessary. In the case a consensus is not possible, a voting system will be implemented where the majority vote will be the result of the decision. It should be noted that the voting system is intended to act as a last resort when attempts at reaching consensus through healthy discussion have been exhausted.